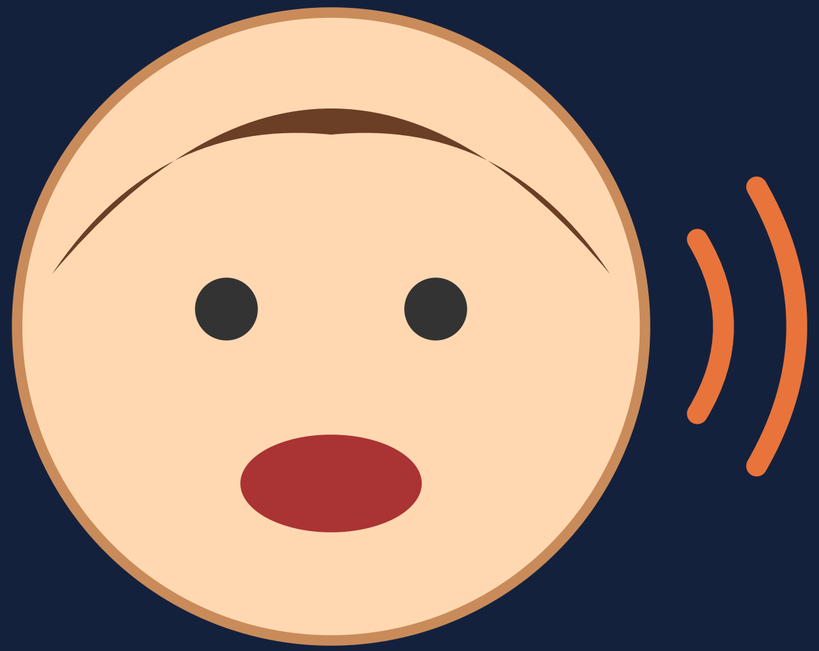
LAB E · FURHAT GENAI LAB

Voice RAG Avatar

Talk to your documents with Streamlit and OpenAI

github.com/iportilla/furhat-rag



THE GOAL

What you’ll build in 45 minutes

Ingest documents

Upload .txt, .md or .pdf files. The app splits and indexes them.

Ask by voice

Record a question. It’s transcribed and matched to your documents.

Hear the answer

An avatar speaks a grounded answer and shows its sources.

You’ll learn: RAG · Embeddings · Speech-to-text · Text-to-speech · Streamlit state

CONCEPTS

Why retrieval matters

Question: **“When are you open on weekends?”**

WITHOUT RAG

“Most flower shops open around 9am on Saturdays…”

A confident guess. The model has never seen your shop.

WITH RAG

“We’re open 10am to 4pm on weekends, according to flower_shop.txt.”

Grounded in your document, and it names the source.

RAG = find the relevant pieces of your documents, then let the model answer **only** from them.

CONCEPTS

The big picture

1 · INGEST, ONCE PER UPLOAD



2 · ASK, EVERY QUESTION



Retrieve searches the **Index** built in step 1. That link is the whole trick.

CONCEPTS

How the app finds the right chunks

An **embedding** is a list of numbers that captures what a text means. Similar meaning gives vectors that point the same way.

1. Ingest stores one vector per chunk.
2. Your question becomes a vector the same way.
3. **Cosine similarity** scores every chunk; the top 4 go to the model.

"OPEN ON WEEKENDS?"

Score	Chunk	Result
0.82	shop hours	Used
0.47	delivery terms	Used
0.31	rose care	Used
0.12	tulip season	Not used

Illustrative scores. In code: vectors @ question in top_k().

INSIDE THE APP

One question, step by step

STEP 1

Record

You click the mic and ask. Streamlit’s st.audio_input captures it.

STEP 2

Transcribe

The audio is sent to OpenAI and comes back as text.

STEP 3

Retrieve

The question is embedded and the top 4 chunks are picked.

STEP 4

Answer

Prompt = rules + chunks + recent chat + question.

STEP 5

Speak

The answer text is turned into MP3 audio.

STEP 6

Avatar

The audio plays; the mouth opens with the volume.

INSIDE THE APP

Four OpenAI models, four jobs

Step	Model	Input	Output
Transcribe	gpt-4o-mini-transcribe	Mic audio	Question text
Embed	text-embedding-3-small	Chunks, question	Vectors
Answer	gpt-4o-mini	Question + top 4	Short answer
Speak	gpt-4o-mini-tts	Answer text	MP3 audio

All four names sit at the top of **app.py**. Change one line to try a different model.

INSIDE THE APP

Streamlit re-runs your whole script

You click, type or record → app.py runs again from the top → Only **st.session_state** survives

Kept in state: `chunks` `vectors` `messages` `reply_audio` `last_audio`

That’s why the app hashes each recording: the same clip stays in the mic widget across re-runs and must not be transcribed twice.

INSIDE THE APP

A tour of app.py, top to bottom

Section	Functions	What it does
Settings	CHAT_MODEL, EMBED_MODEL…	Model names, easy to swap
Ingest	read_file, split_chunks, embed	Files → chunks → vectors
Retrieve + answer	top_k, answer	Best chunks → grounded prompt
Voice	transcribe, speak	Speech-to-text, text-to-speech
Avatar	avatar	SVG face + lip-sync JavaScript
UI	end of file	Sidebar, mic, chat history

About 200 lines in one file. Read it in the same order as the pipeline.

HANDS-ON

Setup in six commands

```
git clone https://github.com/iportilla/furhat-rag.git
```

MACOS / LINUX

```
cd furhat-rag
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
streamlit run app.py
```

WINDOWS (POWERSHELL)

```
cd furhat-rag
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
streamlit run app.py
```

HANDS-ON

Your key lives in .env, not in code

.env · next to app.py

```
OPENAI_API_KEY=sk-proj-…
```

1. Create a key at platform.openai.com/api-keys. Copy it: it’s shown once.
2. Copy .env.example to .env and paste the key after the =.
3. Save, then restart the app with Ctrl+C and streamlit run app.py.

Do	Don’t
KEY=sk-…	KEY = sk-… (spaces)
Name it .env	.env.txt or env
Same folder as app.py	In docs/ or a parent
Check git status first	Commit or share it

Windows: Notepad may add .txt. Turn on View › File name extensions to see it.

HANDS-ON

Your turn: run the lab

1 · INGEST

Upload docs/flower_shop.txt and click Ingest.

2 · ASK

Click the mic, ask a question, click stop.

3 · CHECK

Open Sources to see which chunks were used.

4 · FOLLOW UP

Ask “And on Mondays?” to test memory.

Questions to try

- When are you open on weekends?
- Do you deliver, and how much?
- How do I care for roses?
- How much are orchids? (should say “I don’t know”)

GOING FURTHER

Stretch challenges

Give it a personality

Rewrite the system prompt in answer().

Tune retrieval

Change k, chunk_size and overlap. What breaks?

Measure latency

Time each step from question to speech.

Keep the index

Save vectors with np.save; load at startup.

Go offline

Swap OpenAI for Ollama chat and embed.

Put it on Furhat

Send the answer to request_speak_text().

GOING FURTHER

When something breaks

You see	Try this
“Set OPENAI_API_KEY…”	.env missing, misnamed, or in the wrong folder
No module named streamlit	Activate the venv, then pip install again
Mic button does nothing	Allow the mic in the browser; use localhost
Avatar is silent	Autoplay blocked: press ▶ under the face
401 or 429 error	Bad key, or add billing credit
“I don’t know” for known facts	Ingest again after a reload; scanned PDFs have no text

WRAP-UP

Retrieve. Answer. Speak.

Grounded answers from your own documents, by voice, in about 200 lines of Python.

`github.com/iportilla/furhat-rag`

Questions?